
Machine Learning Applied to Thermal and Energy Engineering Systems

Integrated study: genetic algorithms, neural networks, physics-informed loss functions and optimal control

Maxime Denis & Jean Duthoit

ME249 — Machine Learning in Engineering

University of California, Berkeley — Prof. V. Carey — Fall 2025

Abstract

This document presents an integrated synthesis of four projects carried out as part of the ME249 course at UC Berkeley (Fall 2025). Each project applies a distinct machine learning paradigm to a thermal or energy engineering system: (1) genetic algorithm optimization of a nucleate boiling heat flux model using NASA microgravity data, (2) neural networks with manual backpropagation then Keras for a hybrid solar/fossil gas turbine, (3) deep learning with custom loss functions and physics-informed neural networks (PINN) for a photovoltaic panel and 2D heat conduction, and (4) multi-class classification with dropout and softmax for optimal control of a multi-panel PV system. The whole illustrates a coherent methodological progression — from gradient-free optimization to the direct integration of physical laws into the loss function — while critically analyzing inherent limitations: overfitting, hyperparameter sensitivity, data scarcity.

Table of Contents

1. General Introduction
2. Project 1 — Nucleate Boiling in Microgravity (Genetic Algorithm)
 - 2.1 Physics of the System
 - 2.2 Empirical Model and Genetic Algorithm
 - 2.3 Results and Physical Interpretation
 - 2.4 Sensitivity to Initial Conditions

3. Project 2 — Hybrid Gas Turbine (Neural Networks)
 - 3.1 Physics of the System
 - 3.2 Network Architecture and Backpropagation
 - 3.3 Numerical Results
 - 3.4 Comparison: Gradient Descent vs. Genetic Algorithm
4. Project 3 — PV Panel: Deep Learning and PINN
 - 4.1 Physics of the Photovoltaic Panel
 - 4.2 Normalization, Architecture, Results
 - 4.3 Custom Loss Functions
 - 4.4 Physics-Informed Neural Network (PINN) — 2D Heat Conduction
5. Project 4 — Multi-Panel PV System: Classification and Control
 - 5.1 Three Wiring Modes
 - 5.2 Continuous Regression and Overfitting
 - 5.3 One-Hot Classification, Softmax and Dropout
6. Cross-Cutting Synthesis and Critical Discussion
7. Conclusion

1. General Introduction

Thermal and energy engineering produces systems whose behavior is governed by nonlinear partial differential equations, multiphase transfer phenomena, and multi-scale couplings. The associated experimental data are costly to obtain, noisy, and fundamentally multivariate. In this context, machine learning (ML) provides a methodological framework for extracting robust predictive models from limited datasets.

This document synthesizes four projects from the ME249 course, each applying a distinct ML paradigm to a real engineering system:

- Project 1 — Genetic algorithm: fitting a nucleate boiling model in microgravity using NASA data.
- Project 2 — Neural networks: manual backpropagation then Keras for a hybrid solar/fossil gas turbine.
- Project 3 — Deep learning and PINN: PV panel and 2D heat conduction with physical constraints.
- Project 4 — Classification and optimal control: multi-panel PV system with dropout and softmax.

The common thread is a methodological progression: each project extends the capabilities of the previous one, from gradient-free optimization to the direct integration of physical laws into the loss function, reflecting the historical evolution of ML in engineering.

2. Project 1 — Nucleate Boiling in Microgravity

2.1 Physics of the System

Nucleate boiling is a high-flux heat transfer mechanism: vapor bubbles form on a heated surface, grow, and detach, carrying latent heat away. Under terrestrial gravity, buoyancy drives detachment. In microgravity ($g \rightarrow 0$), this mechanism disappears — bubbles remain attached and form an insulating vapor film that can trigger critical heat flux (CHF).

For water/2-propanol binary mixtures, a phenomenon compensates for this loss: the Marangoni effect. 2-propanol, being more volatile, evaporates preferentially at the bubble interface, creating a concentration gradient $\rightarrow$ surface tension gradient $\rightarrow$ convection directed toward the heated surface. This effect sustains a finite heat flux even at $g \rightarrow 0$. The key parameter is:

$$\gamma = (\sigma_{\text{water}} - \sigma_{\text{mixture}}) / \sigma_{\text{mixture}}$$

The data come from NASA parabolic flight campaigns aboard the KC-135 aircraft ("vomit comet"), with g ranging from $0.01 g_{\text{en}}$ to $2 g_{\text{en}}$, pressure 5.5–9.5 kPa.

2.2 Empirical Model and Genetic Algorithm

Two power-law models are fitted to the data. The 3-parameter model:

$$q'' = n_1 \cdot (T_w - T_{\text{sat}})^{n_2} \cdot g^{n_3} \quad (\text{Eq. 1.1})$$

The 5-parameter model incorporates the Marangoni effect and pressure:

$$q'' = n_1 \cdot (T_w - T_{\text{sat}})^{n_2} \cdot (g + n_4 \cdot g_{\text{en}} \cdot \gamma)^{n_3} \cdot P^{n_5} \quad (\text{Eq. 1.5})$$

The logarithmic transformation linearizes the problem: $\ln(q'') = \ln(n_1) + n_2 \cdot \ln(\Delta T) + n_3 \cdot \ln(g)$. The normalized error function is:

$$F_{\text{err}} = \sum |f_{\text{err},i}| / |\ln(q''_{\text{data},i})| \quad (\text{Eq. 1.3})$$

Genetic algorithm mechanism: population of N_s solutions (vectors $[n_1, \dots, n_k]$). At each generation: fitness evaluation ($= -F_{\text{err}}$), selection of the n_{keep} best, crossover (random gene-wise inheritance, $p = 0.5$), mutation ($\pm 9\%$ random on each constant, adaptive amplitude).

2.3 Results and Physical Interpretation

Parameter	Optimized Value	Physical Interpretation
n_1 (prefactor)	1.15×10^{-3}	Flux amplitude — fluid/surface properties
n_2 (ΔT exponent)	3.10	Strong thermal nonlinearity (consistent with Rohsenow: $q'' \sim \Delta T^3$)
n_3 (g exponent)	0.062	Very low: Marangoni effect dominates buoyancy
F_{err}	< 0.035	Normalized error — target achieved
RMS	$\approx 37 \text{ W/cm}^2$	$\sim 15\text{--}20\%$ relative error — comparable to $\pm 10\%$ measurement uncertainty

Table 1 — Optimized parameters of the 3-parameter model (genetic algorithm).

The exponent $n_2 \approx 3.1$ on wall superheat is consistent with classical correlations (Rohsenow, Kutateladze). The exponent $n_3 \approx 0.062$ is remarkably low: for a pure fluid, n_3 would be ~ 0.25 for $g > 0.1g_{\text{en}}$. The reduction to 0.062 quantitatively confirms that the Marangoni effect largely compensates for the loss of buoyancy — a result directly useful for the design of space thermal systems.

With the 5-parameter model, the term $(g + n_4 \cdot g_{\text{en}} \cdot \gamma)$ explicitly captures the transition between the gravitational and Marangoni regimes. Convergence to $n_4 \approx 1.054$ indicates that the Marangoni term is equivalent to approximately one terrestrial gravitational acceleration, making it the dominant mechanism at $g < 1g_{\text{en}}$.

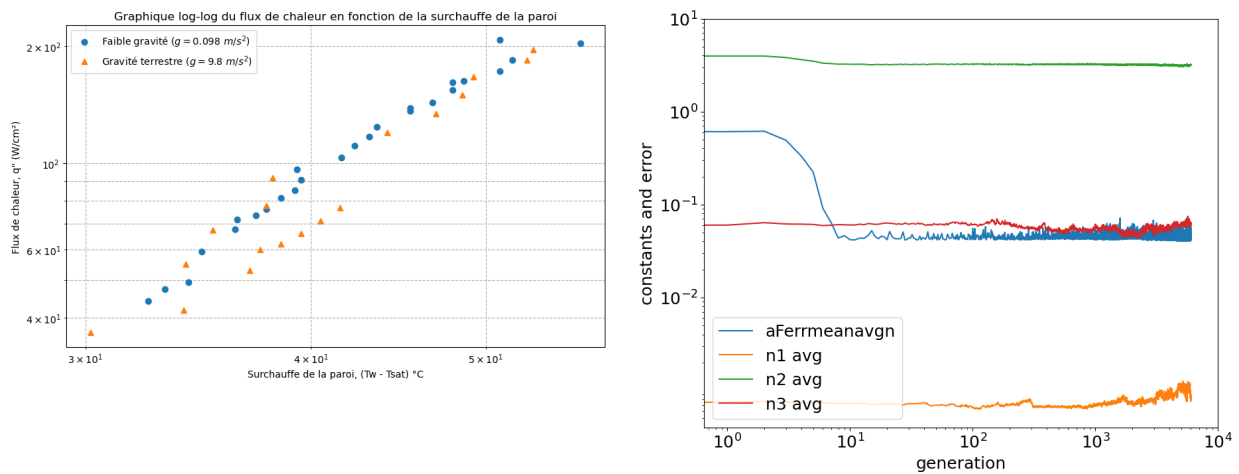


Figure 1a — q'' vs ΔT (log-log) for two gravity levels. | Figure 1b — Genetic algorithm convergence: evolution of constants and error.

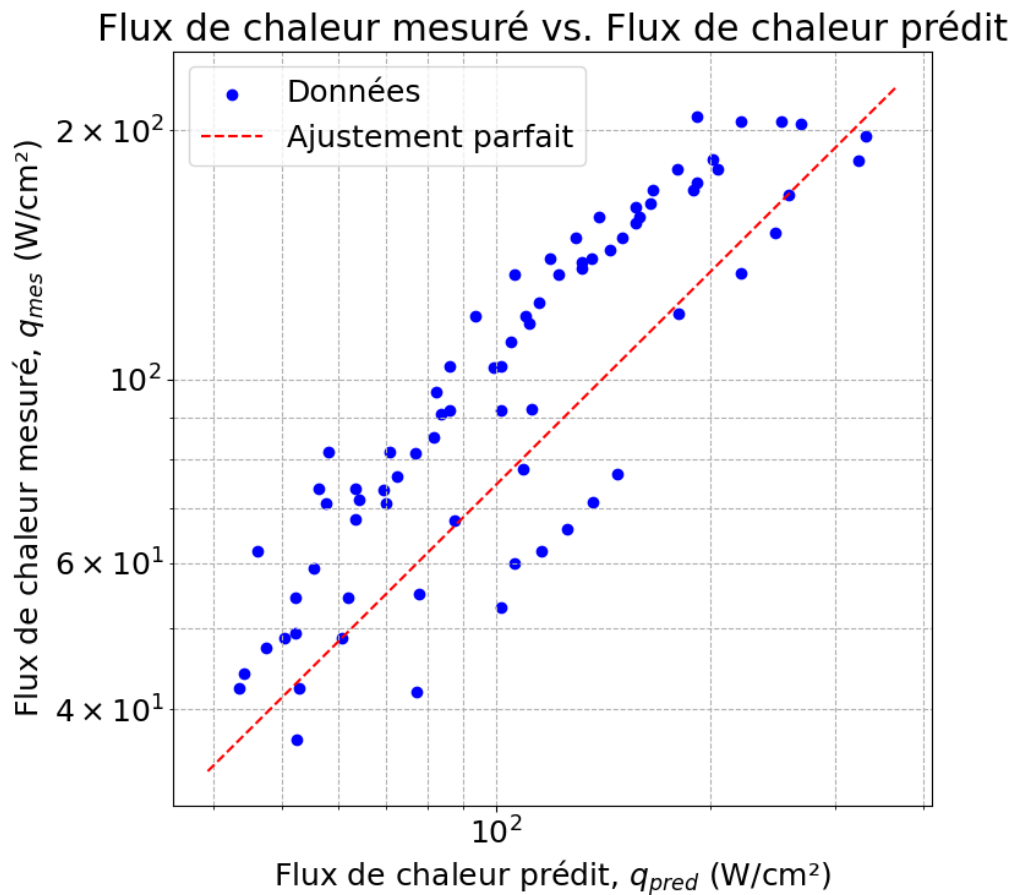


Figure 2 — Measured flux vs predicted flux (log-log). Points aligned on the diagonal confirm the quality of the genetic algorithm fit.

2.4 Sensitivity to Initial Conditions

Four sets of initial conditions were tested. The genetic algorithm is relatively insensitive to initialization thanks to the large mutation mechanism ($\pm 9\%$). Very distant initializations require more generations but converge to the same optimal values, validating the robustness of the identified minimum. The key Python code for the genetic loop:

```
# Selection + crossover + mutation for j in range(NS - nkeep): nmate1 =
random.randint(0, nkeep-1) nmate2 = random.randint(0, nkeep-1) for k in
range(len(n[0])): if random.random() < 0.5: ntemp[nkeep+j][k] = n[nmate1][k] * (1 +
0.09*2*(0.5-random.random())) else: ntemp[nkeep+j][k] = n[nmate2][k] * (1 +
0.09*2*(0.5-random.random()))
```

3. Project 2 — Hybrid Solar/Fossil Gas Turbine

3.1 Physics of the System

Project 2 models two systems. Part 1 (Tasks 1.1/1.2) uses a generic neural network with 3 inputs — temperature T (°C), relative humidity (~13–15%), pressure (~306–311 kPa) — to predict the wet-bulb temperature α (~30–47 °C). This is the reference case for comparing manual backpropagation vs Keras.

Part 2 (Tasks 2.1–2.4) models a hybrid solar/fossil gas turbine. A solar heat exchanger and a fossil fuel burner are mounted in series. Inputs: T_1 (air inlet temperature, K), γ (solar energy fraction, dimensionless), Q_s (solar flux, kW). Output: α (system efficiency/performance, dimensionless). Normalization median values: $T_{\text{med}} = 295.5$ K, $\gamma_{\text{med}} = 0.25$, $Q_{s,\text{med}} = 1500$ kW, $\alpha_{\text{med}} = 61.19$.

3.2 Network Architecture and Backpropagation

Sequential 3-layer network — forward computation:

$$\begin{aligned} z_1 &= w_{01} \cdot x_{01} + w_{02} \cdot x_{02} + w_{03} \cdot x_{03} + b_1 \rightarrow a_1 = \sigma(z_1) \\ z_2 &= w_{12} \cdot a_1 + b_2 \rightarrow a_2 = \sigma(z_2) \\ z_3 &= w_{23} \cdot a_2 + b_3 \rightarrow a_3 = \sigma(z_3) \end{aligned}$$

ELU activation function: $\sigma(z) = z$ if $z \geq 0$; $\exp(z)-1$ if $z < 0$. Avoids the "dead neuron" problem of ReLU (zero gradient for $z < 0$).

Error and update (backpropagation) — full chain rule for w_{01} :

$$\begin{aligned} \partial E_3 / \partial w_{01} &= 2(a_3 - y_3) \cdot \sigma'_3(z_3) \cdot w_{23} \cdot \sigma'_2(z_2) \cdot w_{12} \cdot \sigma'_1(z_1) \cdot x_{01} \text{ (Eq. 2.1)} \\ w_{01,\text{new}} &= w_{01} + \gamma \cdot (0 - E_3) / (\partial E_3 / \partial w_{01}) \text{ with } \gamma \approx 1/(\text{number of weights+biases}) \end{aligned}$$

3.3 Numerical Results

Task 1.1 — First Principles vs Keras Comparison:

Parameter	First Principles (Maxime Denis)	Keras
w_{01}	1.2223	1.2129
w_{02}	0.3903	0.3517
w_{03}	0.6905	0.7118
b_1	-0.1595	-0.1449
w_{12}	0.7172	0.7048
b_2	-0.1268	-0.1102
w_{23}	0.6970	0.6805
b_3	0.0052	0.0245
Error	RMS = 0.0181	MAE = 0.0133

Table 2 — Converged weights and biases: First Principles vs Keras (actual obtained values). Differences are on the order of 10^{-2} to 10^{-3} , validating the manual implementation.

Impact of initialization (+20%): the First Principles model no longer converges (RMS $\approx$ 11% — exit condition never reached), while Keras remains stable with MAE < 0.014 . This demonstrates the robustness of the RMSprop optimizer compared to the simple gradient descent of the manual implementation.

Task 2.2 — Hybrid turbine Keras: loss converged to $0.0227 < 0.04$ (target achieved). The surface $\alpha(T_1, Q_s)$ shows monotonic growth with solar flux and weak dependence on T_1 , as physically expected.

Task 2.3 — Generalization: RMS = 4.428 on test data. The gap with training error reveals slight under-representation of some regions of the input space.

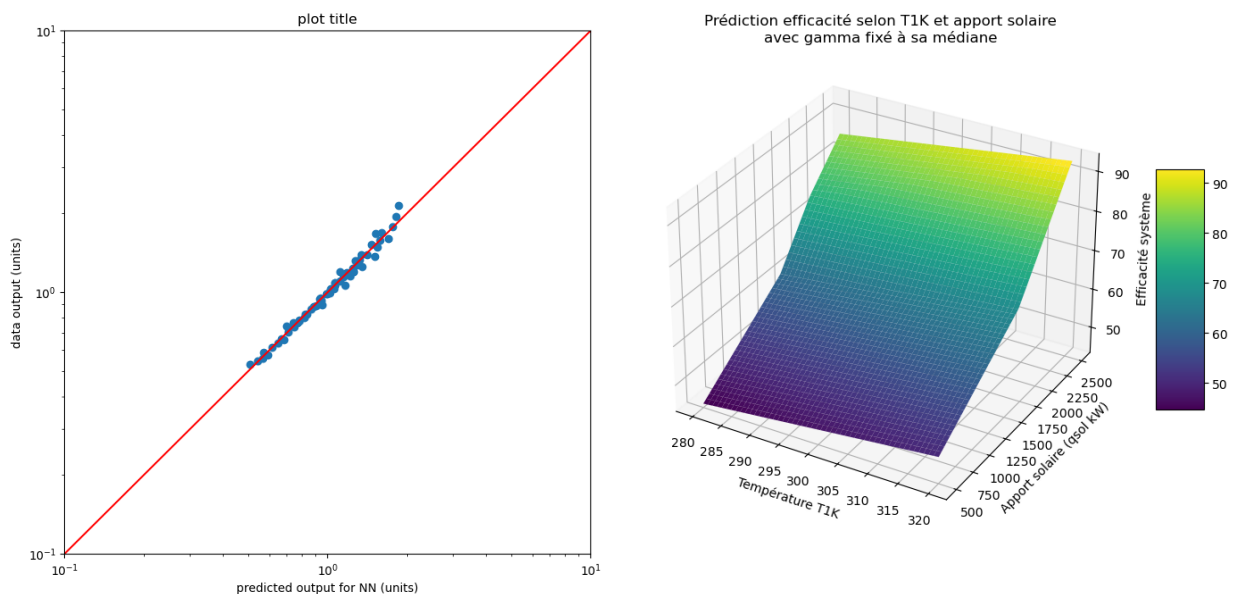


Figure 3a — Predicted vs measured (hybrid turbine, Keras). | Figure 3b — Surface $\alpha(T_1, Q_s)$ at fixed $\gamma = 0.25$: expected growth with solar flux.

Task 2.4 — Weight optimization via PyGAD:

Case	Architecture	Activation	w,b min/max	MAE (fitness)
1	[4,8,4,2]	ReLU	-0.9 / 0.9	0.0748
2	[4,8,4,2]	ELU	-0.9 / 0.9	0.1201
3	[4,8,4,2]	ELU	-2.9 / 2.9	0.0862
4	[4,8,4,2]	ELU	-0.4 / 0.4	0.0904
5	[4,5,4,3,2]	ReLU	-2.9 / 2.9	0.0987
6	[4,5,4,3,2]	ELU	-0.9 / 0.9	0.1117

Table 3 — Comparison of 6 PyGAD configurations. Case 1 (ReLU, [4,8,4,2], ± 0.9) achieves the best MAE of 0.0748.

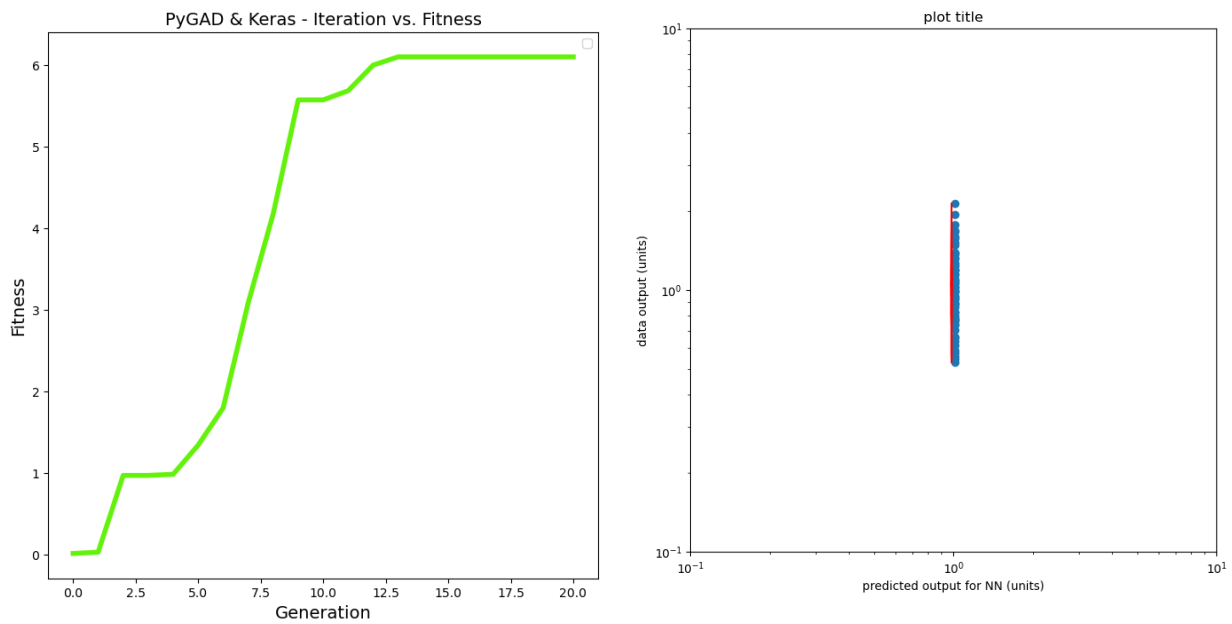


Figure 4a — Fitness vs generations (PyGAD): rapid plateau after ~10–20 generations. | Figure 4b — Predicted vs measured (best PyGAD case).

3.4 Comparison of Two Optimization Paradigms

Criterion	Gradient Descent (Keras)	Genetic Algorithm (PyGAD)
Convergence	Fast (~600 epochs)	Slow (~100 generations, rapid plateau)
Robustness	Sensitive to local minima	Explores disconnected regions
Differentiability	Required	Not required
Best MAE	0.0133	0.0748
Scalability	Excellent	Limited — $O(N_S \times D)$

Table 4 — Comparison of gradient descent vs genetic algorithm.

4. Project 3 — PV Panel: Deep Learning and PINN

4.1 Physics of the Photovoltaic Panel

The PV panel studied has 72 cells in series, each with an area of 173 cm^2 . The photo-generated current is $I_v = q_e \cdot \phi_g \cdot A$, where q_e is the electron charge, ϕ_g the photon flux and A the active area. The output voltage obeys $V_L = I_L \cdot R_L$. The delivered power is $\dot{W} = V_L \cdot I_L = I_L^2 \cdot R_L$.

In series, the total voltage is the sum of individual voltages, and the current is common. The I–V characteristic of the panel shows a current source region (quasi-flat) and a rapid drop at high voltage. The maximum power point (MPP) depends on T_{air} , I_D and R_L — this is precisely the object of ML modeling.

4.2 Normalization, Architecture and Results (Task 3.1.1)

Normalization by the median (robust to outliers, unlike the mean) is applied to each parameter. The data (36 points) are randomly split into 2/3 training (24 points) / 1/3 validation (12 points).

Architecture: Dense(6,ELU) → Dense(6,ELU) → Dense(12,ELU) → Dense(6,ELU) → Dense(2, linear). Optimizer RMSprop(lr = 0.0002), 800 epochs, EarlyStopping.

Dataset	Total MAE	MAE V_L	MAE	Interpretation
Training (24 pts)	0.00263	0.0019	0.0034	Very good fit
Validation (12 pts)	0.01861	0.0084	0.0288	Slight overfitting (×7 — acceptable)

Table 5 — Error metrics Task 3.1.1. The ×7 factor between validation and training is notable but expected for only 36 data points.

The surface $\dot{W}(R_L, I_D)$ predicts a power of 90–184 W over the range 4–8 Ω and 500–1200 W/m². The monotonically increasing dependence on I_D and decreasing dependence on R_L is physically consistent with the I–V characteristic.

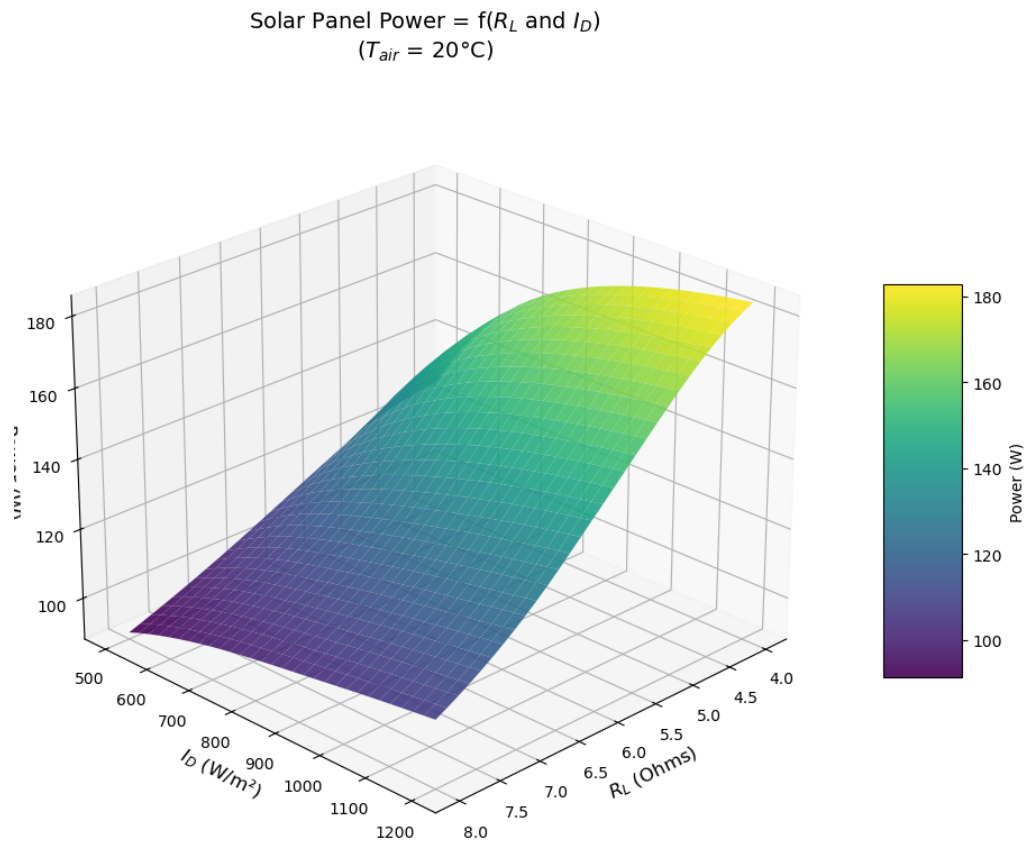


Figure 5 — Surface $\hat{W}(R_L, I_D)$ at $T_{air} = 20^\circ\text{C}$. Power from 90 W ($R_L = 8\ \Omega$, $I_D = 500\ \text{W/m}^2$) to 184 W ($R_L = 4\ \Omega$, $I_D = 1200\ \text{W/m}^2$).

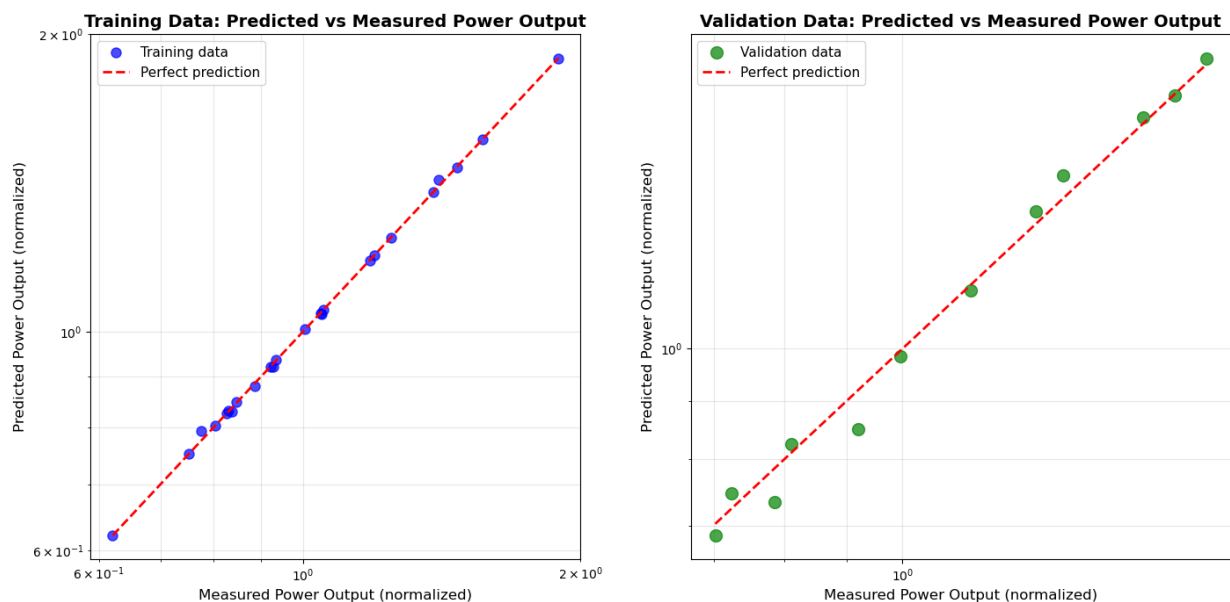


Figure 6a — Predicted vs data (training, log-log): near-perfect alignment. | Figure 6b — Predicted vs data (validation, log-log): slight scatter, correct generalization.

4.3 Custom Loss Functions (Task 3.1.2)

Two loss functions are compared over 6000 epochs:

Function	Formula	Behavior	Error Indicator
MSE (baseline)	$\text{mean}((y_{\text{pred}} - y_{\text{true}})^2)$	Penalizes large errors quadratically	$\text{rms} = \sqrt{\text{best_mse}}$
m4e (custom)	$\text{mean}((y_{\text{pred}} - y_{\text{true}})^4)$	Very strongly penalizes outliers ($\propto^2$)	$s = \sqrt{\sqrt{\text{best_m4e}}}$

Table 6 — Comparison of MSE vs m4e loss. The m4e concentrates correction on the largest residuals.

Python code for m4e:

```
def mean_double_squared_error(y_true, y_pred): return K.mean(K.square(y_pred - y_true) *
K.square(y_pred - y_true)) model.compile(optimizer='rmsprop',
loss=mean_double_squared_error)
```

The m4e can improve robustness to outliers, but at the cost of slower convergence and greater sensitivity to initialization. The optimal choice depends on the residual distribution and tolerance for large point errors.

4.4 Physics-Informed Neural Network (PINN) — 2D Heat Conduction

The second part of Project 3 introduces a conceptual breakthrough: instead of approximating an input-output relationship from data, the network approximates the solution of a PDE — the Laplace equation in a 2D solid domain (3×5 cm):

$$\partial^2 T / \partial x^2 + \partial^2 T / \partial y^2 = 0 \text{ in the interior domain (Eq. 4.1)}$$

Temperatures are measured only on the perimeter (from 10 °C at the upper right corner to 50 °C at the lower edge). No interior measurements. The physics alone constrains the internal field — this is the fundamental value of PINN.

The composite loss function has three terms:

- L_b = boundary residual: $[T(x,y)_{\text{boundary}} - T_{\text{measured}}]^2$ — applied on randomly sampled boundary points.
- L_r = Laplace residual: $[T_{xx}(x,y) + T_{yy}(x,y)]^2$ — computed via automatic differentiation using TensorFlow GradientTape.
- L_{range} = thermodynamic penalty: if $T > T_{\text{max}}$ or $T < T_{\text{min}}$, penalty proportional to $(T - T_{\text{lim}})^2$. Encodes the 2nd law (T bounded by boundary values).

$$L_{\text{total}} = L_b + \kappa \cdot L_r + 20 \cdot L_{\text{range}} \quad (\kappa: \text{adjustable parameter, weight of the Laplace residual})$$

```
def custom_loss_function(kappa, x_max, y_max, epoch_tracker):
    def loss(y_true, y_pred):
        data_loss = K.mean(K.square(y_pred - y_true)) # L_b
        laplace_res = compute_laplace_residual(model, x_max) # L_r via GradientTape
        range_pen = compute_range_penalty(y_pred, T_min, T_max) # L_range
        return data_loss + kappa * laplace_res + 20 * range_pen
    return loss
```

Results: $T(0,0) = 49.997$ °C (measurement: 50 °C), $T(0,0.5) = 50.004$ °C — sub-degree accuracy. Loss < 0.005 achieved in ~200 epochs. The predicted field is validated by comparison with the finite difference solution (Gauss-Seidel) on the same grid.

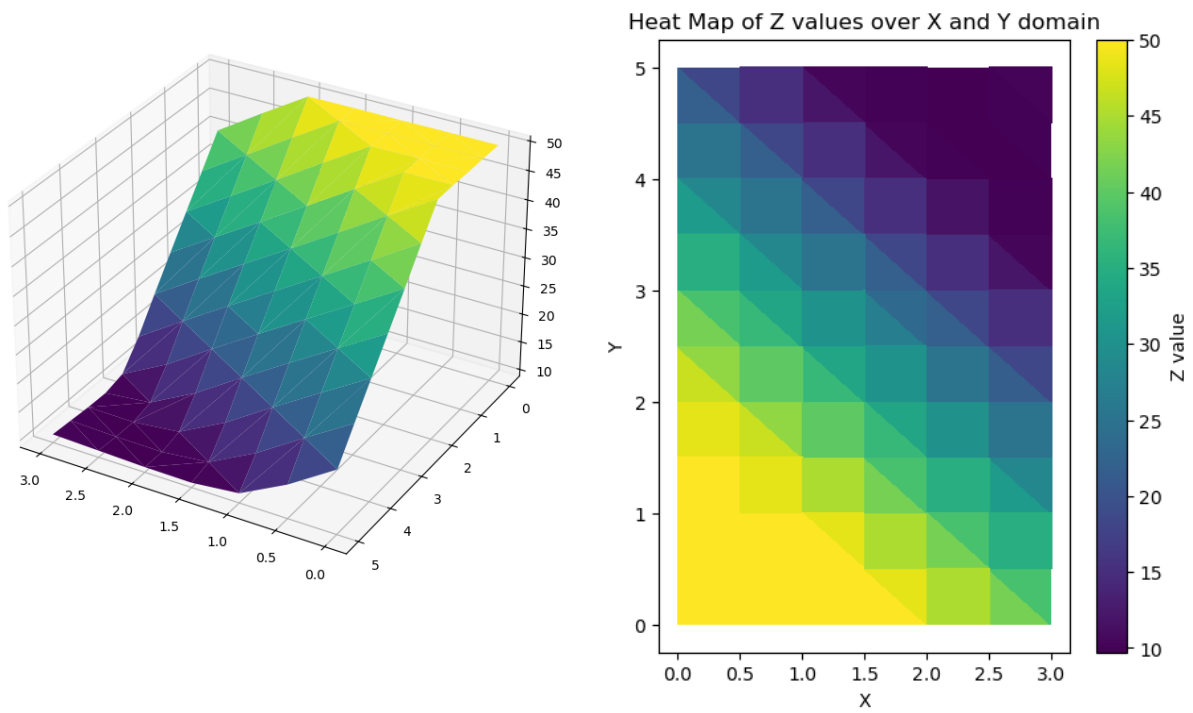


Figure 7a — 3D temperature field predicted by the PINN. | Figure 7b — Heatmap $T(x,y)$: thermal gradient from hot corner (lower left) to cold corner (upper right).

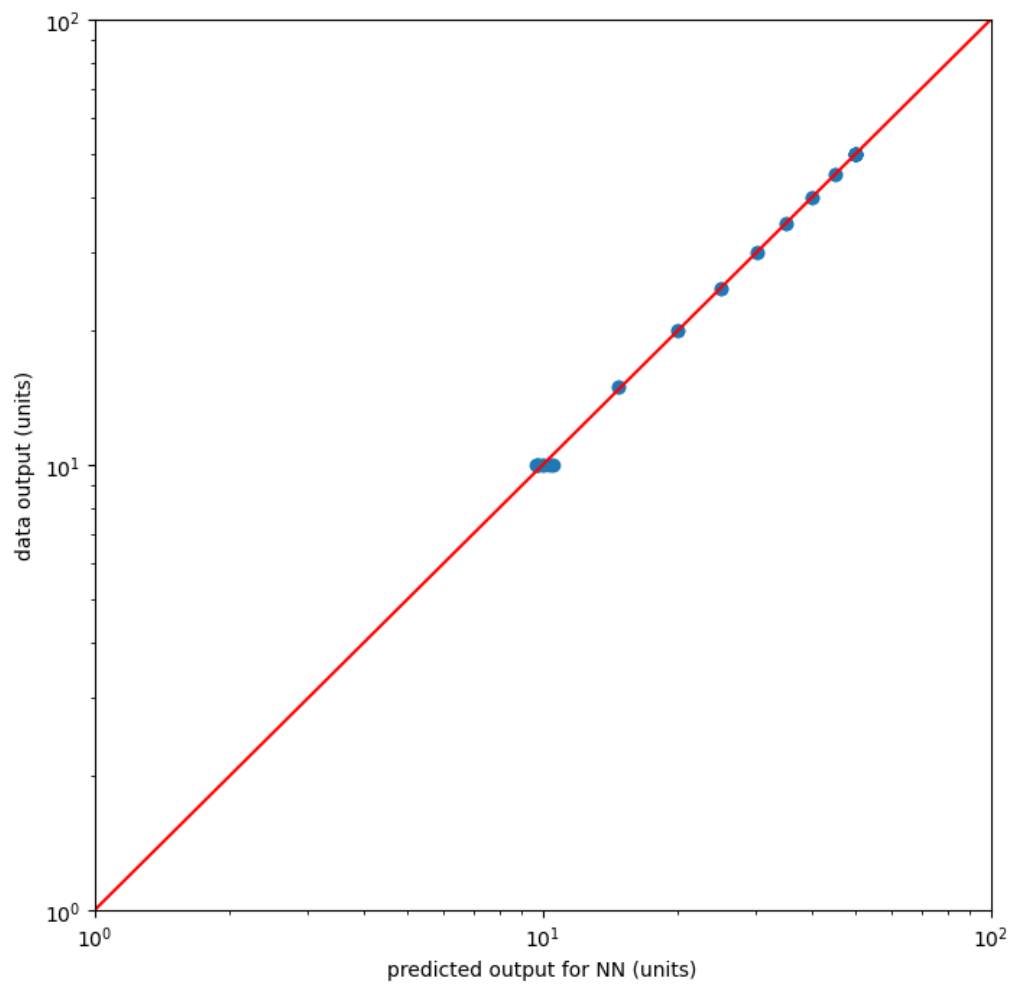


Figure 8 — PINN predictions vs boundary data: near-perfect alignment on the diagonal.

5. Project 4 — Multi-Panel PV System: Classification and Control

5.1 Three Wiring Modes

A system of 4 identical PV panels can be wired in three configurations, producing very different V-I characteristics:

Mode	Configuration	V_{tot}	I_{tot}	Advantage
0	4 in parallel	V_{panel}	$4 \cdot I_{\text{panel}}$	Maximum current — optimal at low R_L
1	2×2 series/parallel	$2 \cdot V_{\text{panel}}$	$2 \cdot I_{\text{panel}}$	Voltage/current trade-off
2	4 in series	$4 \cdot V_{\text{panel}}$	I_{panel}	Maximum voltage — optimal at high R_L

Table 7 — Wiring modes. The power $\dot{W} = V \cdot I$ is theoretically identical for ideal panels, but resistive losses and PN junction nonlinearities mean that the optimal mode depends on conditions (R_L , I_D , T_{air}).

5.2 Continuous Regression and Overfitting (Task 4.2.1)

The network predicts $[V_L, \dot{W}]$ for each mode M (non-normalized input) + normalized conditions. Architecture: 206 trainable parameters, 48 data points, 32 training / 16 validation, 800 epochs.

Metric	Training	Validation	Val/Train Ratio
Normalized MAE ($\dot{W}$)	0.288	0.913	×3.2
Physical MAE (W)	49.1 W	155.7 W	×3.2
Best epoch	775/800	—	Late convergence

Table 8 — Metrics Task 4.2.1. The ×3.2 ratio between validation and training indicates severe overfitting: 206 parameters for 48 data points (ratio 4.3) is structurally underconstrained.

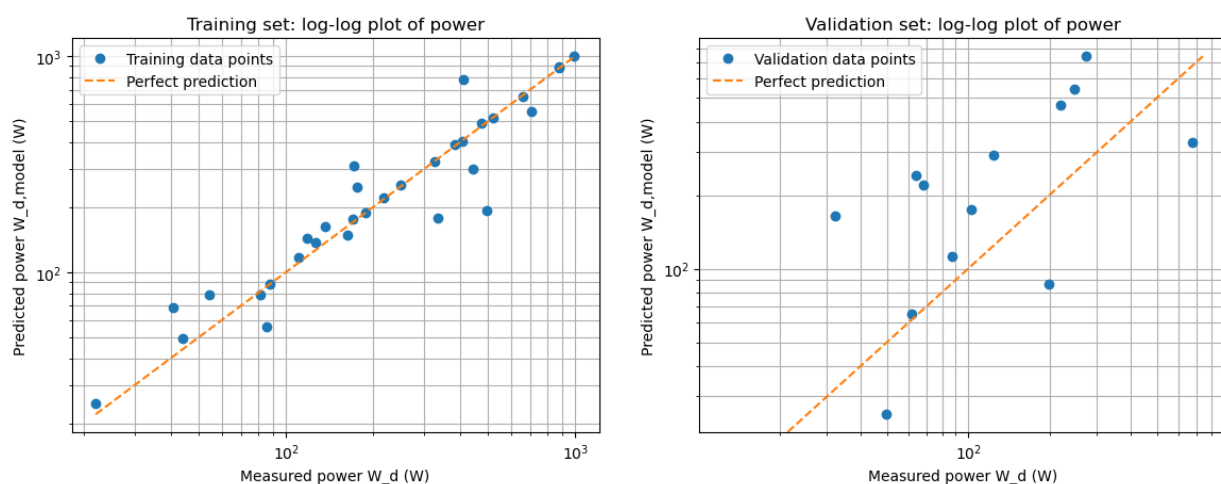


Figure 9a — Predicted vs data (training). | Figure 9b — Predicted vs data (validation): strong scatter, signature of overfitting.

5.3 One-Hot Classification, Softmax and Dropout (Task 4.2.2)

The problem changes nature: from regression (predicting $\hat{W}$) to classification (predicting which mode maximizes $\hat{W}$). One-hot encoding converts discrete classes: mode 0 $\rightarrow$ [1,0,0], mode 1 $\rightarrow$ [0,1,0], mode 2 $\rightarrow$ [0,0,1].

Softmax output function: $\sigma_i = \exp(z_i) / \sum \exp(z_j)$ — normalizes outputs to a probability distribution (sum = 1). Loss: categorical cross-entropy: $L = -\sum y_{\text{true}} \cdot \log(y_{\text{pred}})$. Optimizer: Adam (adaptive learning rate + momentum).

Reading the model output:

```
outpt = model.predict([[T_air, I_D, R_L]]) # outpt[0]  $\approx$  [0.9955, 4.48e-4, 8.57e-9] #  $\rightarrow$ 
probabilities of modes 0, 1, 2 Mmaxint = np.argmax(np.round(outpt[0]))
```

Dropout ($p = 0.25$) is added after each hidden layer to reduce overfitting: each neuron is randomly deactivated during training, forcing redundancy of representations and limiting co-adaptation of weights.

```
model = keras.Sequential([ Dense(13, activation=K.elu, input_shape=[3]), Dense(26,
activation=K.elu), Dropout(0.25), Dense(13, activation=K.elu), Dropout(0.25),
Dense(num_classes, activation='softmax') ])
model.compile(loss=categorical_crossentropy, optimizer=Adam())
```


6. Cross-Cutting Synthesis and Critical Discussion

6.1 Methodological Progression

Project	Physical System	ML Algorithm	Identified Limitation
P1	Microgravity boiling (NASA)	Genetic algorithm	RMS $\approx$ 15–20% — comparable to measurement uncertainty
P2	Hybrid solar/fossil turbine	Manual backprop + Keras + PyGAD	Sensitivity to initial conditions (FP) — slow convergence (PyGAD)
P3	PV panel + 2D conduction	Deep learning + PINN	Moderate overfitting (36 pts) — resolved by PINN on PDE
P4	4-panel PV system	Softmax classification + dropout	Severe overfitting (48 pts, 206 params) — partially mitigated by

Table 9 — Summary of the four projects: systems, methods, and limitations.

6.2 Fundamental Cross-Cutting Concepts

Median normalization. Systematic from Project 2 onward. The median is insensitive to outliers (unlike the mean). The network sees order-1 data, which stabilizes gradients and accelerates convergence.

ELU vs ReLU. ELU ($\exp(z)-1$ for $z < 0$) avoids the dead neuron problem of ReLU (gradient = 0 for $z < 0$) and produces near-zero mean activations, reducing internal covariate shift.

RMSprop vs Adam. RMSprop adapts the learning rate per parameter by dividing by the square root of the running mean square of recent gradients — stable on non-stationary data. Adam adds a momentum term — more aggressive, preferred for classification (P4).

Loss functions and physics. The progression MSE $\rightarrow$ m4e $\rightarrow$ PINN illustrates how to encode physical properties a priori into training. PINN represents the extreme case: physics replaces interior data, invaluable when measurements are scarce.

Overfitting diagnostics. Criterion: $\text{MAE}_{\text{val}} / \text{MAE}_{\text{train}} > 3$ (warning sign). Clearly observed in P4 (ratio 3.2). Remedies explored: dropout (P4), systematic train/val split (P3/P4), EarlyStopping.

7. Conclusion

This ME249 course demonstrates how machine learning can be rigorously and progressively integrated into the training of mechanical engineers. Grounding in the physics of the studied systems — boiling in microgravity, hybrid turbines, PV panels — avoids the trap of purely algorithmic learning and gives meaning to error metrics.

The difficulties encountered — severe overfitting in Project 4, non-negligible RMS error in Project 1 — are not failures but critical lessons: they reveal structural limitations of models (underconstrained, data scarcity) and point toward concrete solutions (regularization, data augmentation, simpler architectures).

The algorithmic progression — genetic algorithm $\rightarrow$ manual backpropagation $\rightarrow$ Keras $\rightarrow$ PINN $\rightarrow$ classification $\rightarrow$ dropout — follows the historical evolution of ML in engineering and offers a coherent panorama of available paradigms for modeling complex physical systems from limited experimental data.

Perspectives: PINN for other PDEs (Navier-Stokes, convection-diffusion), transfer learning between similar PV systems, Bayesian optimization of hyperparameters, integration of thermodynamic constraints into classification architectures.